Supernode Detector: Automated Community Detection for Neuronpedia Attribution Graphs

Supernode Detector: An End-to-End Pipeline for Automated Community Detection, Annotation, and Publication of Sparse Autoencoder Attribution Graphs

Authors: Joseph Lawrence, Konstantinos Krampis

Abstract

Sparse autoencoder (SAE) attribution graphs generated by Neuronpedia's Circuit Tracer typically contain 1,000–1,500 nodes and 40,000–80,000 edges per prompt, making manual interpretation infeasible. Existing community-detection workflows require per-graph algorithm tuning, frequently produce degenerate partitions (either oversized communities that span most of the graph or fragmented partitions of singletons), and offer no integration with downstream annotation or publication. We present **Supernode Detector**, an open-source command-line tool that takes a text prompt and produces an annotated, interactively viewable Neuronpedia graph in a single command. The pipeline (i) generates the attribution graph via the Neuronpedia API, (ii) runs three community-detection algorithms (Louvain, Leiden, and Greedy Modularity) and selects the best partition using a penalty-weighted modularity score that discourages oversized communities and singletons, (iii) recursively splits any community exceeding 30% of the graph at higher Louvain resolutions (capped at 10 sub-communities), (iv) identifies the top-10 critical routing nodes by betweenness centrality and pins them in the viewer, (v) labels each community using Neuronpedia feature explanations with a top-3 keyword summarisation strategy and structural fallback, (vi) trims each community to its top-influence members targeting a configurable fraction (default 15%) of the graph while always retaining bottleneck nodes, and (vii) optionally uploads the result to Neuronpedia via the platform's three-step signed-PUT API after applying the platform-required cantor-pairing transformation to feature identifiers. We validate the tool on three test prompts spanning chemistry, geography, and history domains: a typical 1,200-node graph reduces to 11–14 supernode communities and approximately 13–14% of original nodes in 90 seconds end-to-end, with Louvain consistently selected as the best partition (modularity 0.44–0.50 versus Greedy 0.40–0.42 with only 3 oversized communities). The tool is positioned as an opinionated bridge between low-level circuit-discovery methods (e.g., ACDC, attribution patching, traceback graphing) and the human-readable interactive visualisations that researchers and reviewers actually consume. Source and documentation are available at `github.com/J-Lawrence10/autocircuit` under the MIT license.

Keywords: mechanistic interpretability, sparse autoencoders, attribution graphs, community detection, Neuronpedia, scientific tooling

1. Introduction

Mechanistic interpretability research increasingly relies on **attribution graphs** — directed graphs whose nodes are sparse-autoencoder (SAE) features and whose edges are attention-weighted causal contributions — to localise factual recall, indirect object identification, and other behaviours to specific computational circuits in transformer language models (Olah et al., 2020; Elhage et al., 2021; Meng et al., 2022; Wang et al., 2023). The Neuronpedia platform (Decode Research) has democratised access to such graphs by hosting a Circuit Tracer API that produces them on demand for several open-source models including Gemma-2-2B and Qwen3-4B.

A practical bottleneck blocks the workflow between graph generation and interpretation. A single attribution graph for a typical factual prompt contains roughly 1,200 SAE-feature nodes and 40,000–80,000 weighted edges, far beyond what a human can inspect node-by-node. Researchers must (a) reduce the graph to interpretable supernode clusters, (b) attach human-readable labels to each cluster, (c) format the result as the JSON schema Neuronpedia expects (including a non-trivial cantor-pairing transformation on feature identifiers), and (d) upload the graph through a multi-step signed-URL API. Each of these steps is independently nontrivial, and existing tooling provides no integrated path from prompt to published graph.

This paper introduces **Supernode Detector**, a command-line tool that performs the full workflow in a single invocation. The contributions are:

1. **A penalty-weighted multi-algorithm community selection procedure** that runs Louvain, Leiden, and Greedy Modularity in parallel and selects the partition that maximises modularity subject to penalties for oversized communities and singletons. This avoids the common pathology in which Greedy Modularity wins on raw modularity but produces partitions of two or three giant communities that are useless for interpretation.

2. **An influence-based trimming step** that reduces a 1,200-node clustered graph to approximately 13–15% of its nodes while always retaining critical routing nodes identified by betweenness centrality.

3. **A cantor-pairing export** that automatically transforms raw circuit-tracer feature identifiers to the integer encoding the Neuronpedia viewer requires for feature dashboards to render correctly. This transformation is not documented in the Neuronpedia API and is a frequent source of upload failures for users who attempt manual JSON formatting.

4. **A three-step programmatic upload implementation** of Neuronpedia's `/api/graph/signed-put` → S3 PUT → `/api/graph/save-to-db` flow with automatic slug-uniqueness handling, allowing batch-publication of analyses without web-UI interaction.

5. **End-to-end validation** on three independently chosen factual prompts demonstrating consistent algorithm selection, target trimming ratios, and successful Neuronpedia upload.

The tool is open-source under the MIT license and has been used to generate the supernode visualisations referenced in our companion paper on cross-domain circuit analysis (Lawrence & Krampis, 2026).

2. Related Work

Manual circuit discovery. Early mechanistic interpretability work identifies circuits by hand: the indirect-object identification circuit in GPT-2 Small (Wang et al., 2023), the docstring circuit (Heimersheim & Janiak, 2023), and the factual-association MLPs identified by causal tracing (Meng et al., 2022). These analyses are publication-quality but require weeks of expert effort per circuit.

Automated circuit discovery. ACDC (Conmy et al., 2023) automates circuit discovery by iteratively pruning edges from a hypothesised circuit and measuring task-performance impact. Edge-attribution patching (Syed et al., 2023) and gradient-based attribution methods (Kramár et al., 2024) extend this approach with cheaper approximations to causal effects. These methods produce circuits expressed as edge subsets but do not provide tooling for visualisation, annotation, or publication of the resulting graphs.

SAE feature interpretation. Sparse autoencoders decompose model activations into interpretable features (Cunningham et al., 2024; Bricken et al., 2023; Templeton et al., 2024). Features can be labeled automatically by an LLM examining their top-activating examples (Bills et al., 2023). Neuronpedia (Lin, 2023) hosts SAE feature dashboards with such auto-generated explanations and provides an API for retrieving them.

Interactive visualisation. TransformerLens (Nanda, 2023) provides programmatic access to model internals for exploration. The Neuronpedia graph viewer renders attribution graphs as an interactive force-directed layout with clickable nodes that link to feature dashboards. Anthropic's interactive attribution-graph viewer (Lindsey et al., 2025)

provides similar functionality for Claude. Neither platform provides automated supernode clustering or end-to-end pipelines from prompt to published graph; they expect researchers to bring an already-formatted JSON.

Supernode Detector occupies a niche between automated circuit-discovery methods (which produce edge subsets) and interactive visualisation platforms (which expect formatted graphs). It does not attempt to compete with ACDC or attribution patching for circuit identification; it consumes the output of Neuronpedia's existing Circuit Tracer and focuses on the under-tooled middle steps of clustering, annotation, and publication.

3. Methods

3.1 Pipeline overview

The pipeline consists of seven sequential stages, all implemented in a single Python script (`scripts/supernode_pipeline.py`, ~1,400 lines, MIT license). Stages are: (1) graph generation, (2) multi-algorithm community detection with penalty-weighted selection, (3) recursive splitting of oversized communities, (4) bottleneck identification, (5) semantic labelling, (6) influence-based trimming, and (7) cantor-paired export with optional upload. Each stage can be run independently via command-line flags; the default invocation runs all seven.

3.2 Graph generation

Attribution graphs are generated via the Neuronpedia `POST /api/graph/generate` endpoint with parameters `modelId` (the target model identifier) and `sourceSetName` (the SAE family, defaulting to `gemmascope-transcoder-16k` for Gemma). The API returns a pre-signed S3 URL from which the raw graph JSON is downloaded. Typical graphs contain 1,000–1,500 nodes (SAE features plus embedding and error nodes) and 40,000–80,000 edges. The `--raw-graph PATH` flag bypasses generation and consumes a previously cached graph, which is useful for reanalysis.

3.3 Multi-algorithm community detection with penalty-weighted selection

The graph is converted to an undirected NetworkX graph (edge weights are absolute values of attribution weights). Three community-detection algorithms are run independently:

- **Louvain** (Blondel et al., 2008) via `python-louvain` with `random_state=42`, default resolution 1.0.
- **Leiden** (Traag et al., 2019) via `leidenalg` with the CPM partition type and resolution 0.01 (chosen empirically for sparse attribution graphs).
- **Greedy Modularity** via `networkx.algorithms.community.greedy_modularity_communities`.

Each partition is scored using:

```
score(alg) = modularity(alg) − 0.05 × n_oversized − 0.001 × n_singletons
```

where `n_oversized` is the number of communities containing more than 30% of total nodes and `n_singletons` is the number of communities containing fewer than 3 nodes. The algorithm with the highest score is selected. The penalty weights were chosen empirically: a single oversized community costs 0.05 in score, comparable to a meaningful modularity gap (≈0.05 between competing algorithms in our validation runs), while singletons each cost only 0.001 to permit small but legitimate communities.

This selection procedure addresses a recurring failure mode in which Greedy Modularity returns 3 giant communities with nominally higher modularity than Louvain's 11–14 balanced communities, despite the former being uninterpretable.

3.4 Recursive splitting

If the selected partition still contains a community larger than 30% of total nodes (uncommon but possible after Louvain selection), that community's induced subgraph is recursively split using Louvain at progressively higher resolutions {2.0, 3.0, 4.0, 6.0} until either the largest sub-community falls below 30% or the resolution sweep is exhausted. To prevent fragmentation, the split is capped at 10 sub-communities; if exceeded, the resolution is reduced via a fallback sweep over {1.5, 2.0, 2.5}.

3.5 Bottleneck identification

Betweenness centrality is computed across the full graph (NetworkX's default Brandes algorithm with edge weight). The top-10 nodes by betweenness are identified as bottleneck nodes and stored in `qParams.pinnedIds`, which causes them to be highlighted in the Neuronpedia viewer. Communities containing one or more bottleneck nodes are flagged in the report as "bottleneck communities."

3.6 Semantic labelling

For each community, the top-3 nodes by influence (where influence is the activation × edge-weight product computed during traceback graphing; see companion paper) are queried against the Neuronpedia feature explanation API at `/api/feature/{model}/{layer}-{source_set}/{np_id}`. Two labelling strategies are computed in parallel:

- **Top-3 summarisation:** keyword overlap across the three explanations is extracted via stop-word-filtered tokenisation; the most-frequent shared keywords (≥2) form the label.
- **Top-1 direct:** the single highest-influence feature's explanation is truncated to 35 characters and prefixed with the community's layer range.

A heuristic selects whichever label is more specific (longer non-trivial overlap wins; ties go to top-1). When no explanations are available (typical for Qwen features as of writing) a structural fallback produces labels of the form "L12-L18 cluster (42 nodes)".

3.7 Influence-based trimming

Raw community partitions include low-influence noise nodes that dilute interpretation when rendered. The trimming step applies a two-pass procedure parameterised by `target_pct` (default 0.15):

1. Compute the global node budget: `target_count = target_pct × n_clustered_nodes`.
2. Distribute the budget proportionally to community size, with a per-community minimum of `max(1, min(3, int(avg_budget × 0.3)))` to prevent small communities from being eliminated.
3. Within each community, retain the top-influence nodes until the budget is filled.
4. Bottleneck nodes from §3.5 are always retained regardless of community budget.

In practice this reduces a ~1,200-node graph to ~170 retained nodes (13–14% of the original) while preserving all critical routing structure.

3.8 Cantor-paired export and Neuronpedia upload

The Neuronpedia viewer expects each node's `feature` field to be a single integer encoding both the layer number and the SAE dictionary index using the Cantor pairing function:

```
sae_index   = raw_feature_id mod 16384
cantor(L,i) = (L + i) × (L + i + 1) ÷ 2 + i
```

This transformation is applied to all node `feature` fields during export. Without it, the Neuronpedia viewer renders the graph but cannot resolve feature dashboards when a node is clicked — a common failure mode when users construct annotated JSON manually.

The annotated JSON is written with `qParams.supernodes` (per-community label and member node IDs), `qParams.pinnedIds` (top-10 bottleneck nodes), and `metadata.feature_details.neuronpedia_source_set` (which directs the viewer to the correct SAE family for feature dashboards).

When `--upload` is set, the tool executes Neuronpedia's three-step graph upload:

1. **Signed PUT request.** `POST /api/graph/signed-put` with the JSON file size and content type returns a pre-signed S3 URL and a `putRequestId`. The graph slug has a Unix timestamp appended to guarantee uniqueness and avoid 400 errors from collisions with other users' graphs.

2. **S3 upload.** The cantor-paired JSON is PUT directly to the pre-signed URL (no authentication required for this step).

3. **Database registration.** `POST /api/graph/save-to-db` with the `putRequestId` registers the graph and returns a viewable URL on `neuronpedia.org`.

Authentication uses the `x-api-key` header throughout. The viewable URL and putRequestId are saved to `upload_result.json` next to the annotated graph.

3.9 Outputs

Each invocation produces four files in `output/{model}_{prompt-slug}/`:

- `annotated_graph.json` — the cantor-paired, supernode-annotated graph
- `supernode_report.md` — a markdown report listing communities, algorithm comparison, and bottleneck pins
- `community_visualization.png` — a force-directed layout coloured by community
- `raw_graph.json` — the unmodified original API response (cached for reanalysis)

If `--upload` is used, a fifth file `upload_result.json` records the viewable URL.

4. Experimental Setup

We validate the tool on three independently chosen prompts spanning chemistry, geography, and history — the three knowledge domains studied in our companion paper. All runs use Gemma-2-2B with the `gemmascope-transcoder-16k` SAE family. Each prompt is run with default parameters (no per-prompt tuning) and produces a complete pipeline output.

Validation prompt	Target output	Domain
"The chemical symbol for water is"	"H■O" / "Hydrogen"	Chemistry
"The capital of Australia is"	"Canberra"	Geography
"The Titanic sank in the year"	"1912"	History

The three prompts were chosen to cover: a chemistry prompt outside the 10-element set used in the companion paper (water rather than gold/iron/etc.); a geography prompt with a less canonical answer than the most common templates (Canberra rather than Paris/Tokyo); and a history prompt matching the companion paper's expanded steering target (Titanic). The Australia and water prompts had no cached graphs prior to the experiment; the Titanic graph was previously generated as part of the steering experiments and was rerun to verify reproducibility.

The Australia run was executed with the `--upload` flag to validate the full publication pipeline including programmatic upload.

5. Results

5.1 Cross-prompt summary

All three prompts completed end-to-end in under 100 seconds wall-clock time, including 15–60 seconds for graph generation (variable due to Neuronpedia rate limits) and 30–60 seconds for semantic labelling API calls. The pipeline produced 11–14 supernode communities per graph and retained 13.0–14.4% of original nodes after influence trimming, in close agreement with the 15% target.

Prompt	Raw nodes	Edges	Algorithm chosen	Modularity	Communities	Retained nodes	% retained
Water (chemistry)	1,316	67,343	Louvain	0.4498	14	178	13.5%
Australia (geography)	1,012	40,665	Louvain	0.4570	14	132	13.0%
Titanic (history)	1,314	47,047	Louvain	0.4917	11	172	14.4%

In all three cases Louvain was selected as the best partition. Greedy Modularity nominally produced higher modularity on the water prompt before penalty weighting (0.397 vs 0.450) but did so with only 3 communities, two of which exceeded the 30% threshold; the penalty score correctly down-weighted this partition. Leiden's CPM partition consistently produced the lowest modularity (0.28–0.30) and was never selected.

5.2 Validation prompt 1: chemistry ("The chemical symbol for water is")

The water prompt produced the densest graph in the validation set (67,343 edges from 1,316 nodes, 51 edges per node on average). After filtering embedding and error nodes, the clustered graph contained 1,205 nodes which Louvain partitioned into 14 communities. Three communities were flagged as bottleneck-carrying (containing one or more of the top-10 betweenness nodes), spanning layers L0–L6, L0–L18, and L0–L25. The largest community (community 4, "L0-L18 cluster, 351 nodes, bottleneck") spans 56% of the clustered graph and triggered the recursive-splitting path; after splitting at resolution 2.0, the partition retained 14 communities with no community exceeding 30% of the graph.

Semantic labelling was unable to extract human-readable labels for this graph: 14 of 14 communities received structural fallback labels of the form "L0-L18 cluster (351 nodes)". The cause is sparse Neuronpedia explanations for water-related Gemma SAE features at the relevant layers; this is a known limitation of the current SAE annotation coverage rather than a tool failure.

After influence trimming, 178 nodes were retained (13.5% of clustered, target 15%). The full pipeline took 5 minutes 7 seconds.

5.3 Validation prompt 2: geography ("The capital of Australia is")

The Australia prompt produced the smallest graph (1,012 raw nodes, 40,665 edges). Louvain partitioned the 917-node clustered graph into 14 communities with modularity 0.457, narrowly preferred over Greedy's 3-community solution (modularity 0.425) by the penalty score. No community exceeded the 30% threshold so recursive splitting was not invoked.

The bottleneck identification flagged 3 communities as bottleneck-carrying. The top-5 bottleneck nodes by betweenness centrality were:

Node	Layer	Betweenness	Community
L2_F70110558	2	0.0204	7 (205-node bottleneck cluster)
L6_F10902108	6	0.0125	4 (L3-L23 bottleneck cluster)
L18_F5686859	18	0.0120	5 (L0-L27 bottleneck cluster)
L7_F244642	7	0.0102	4
L21_F17793573	21	0.0095	8

Semantic labelling produced one meaningful label ("the word 'is' (L0-L6)" for community 0); the remaining 13 communities received structural fallbacks. Influence trimming retained 132 nodes (13.0% of clustered).

The `--upload` flag triggered the three-step Neuronpedia upload, which completed in under 5 seconds: signed-PUT request returned a pre-signed S3 URL and `putRequestId`, the cantor-paired JSON was uploaded to S3, and `save-to-db` registered the graph. The viewable URL is `https://www.neuronpedia.org/gemma-2-2b/graph?slug=thecapitalofaust-1776888180439-1776888251`. End-to-end wall time including upload was 1 minute 31 seconds.

5.4 Validation prompt 3: history ("The Titanic sank in the year")

The Titanic prompt was previously analysed during steering experiments but was rerun to verify reproducibility after the cantor-pairing fix. Louvain partitioned the 1,193-node clustered graph into 11 communities with modularity 0.492 — the highest modularity in the validation set. The relatively low community count (11 vs 14 for the other prompts) reflects the more focused circuit structure typical of well-known historical-date prompts.

Two communities were flagged as bottleneck-carrying. Semantic labelling succeeded for two communities ("words indicating periods or divisions" and "the phrase 'in order to'"), the highest semantic-label rate in the validation set. Influence trimming retained 172 nodes (14.4% of clustered).

After cantor-pairing was applied to all 1,307 node features (verified by spot-checking that `cantor(0, 4752) = 11297880` matches the first node), the graph uploaded successfully to `https://www.neuronpedia.org/gemma-2-2b/graph?slug=titanic-sank-supernodes-1775847434`.

5.5 Algorithm-selection behaviour

Across all three validation prompts, the penalty-weighted selector preferred Louvain over Greedy in every case despite Greedy's competitive modularity:

Prompt	Louvain (mod / k)	Leiden (mod / k)	Greedy (mod / k)	Greedy oversized?
Water	0.450 / 14	0.296 / 5	0.397 / 3	Yes (2 communities)
Australia	0.457 / 14	0.294 / 5	0.425 / 3	Likely (≥1 community)
Titanic	0.492 / 11	0.289 / 5	0.484 / 3	Yes (≥1 community)

Without penalty weighting, Greedy would have been selected on the Titanic prompt by raw modularity (0.484 vs 0.492 difference is within typical run-to-run variance), producing 3 giant communities of which two would have spanned more than 30% of the graph. The penalty term `−0.05 × n_oversized` cost Greedy 0.10 and 0.05 in score for the water and Australia/Titanic prompts respectively, restoring Louvain as the choice.

5.6 Trimming consistency

The influence-based trimming step retained between 13.0% and 14.4% of clustered nodes across the three prompts, all within 2 percentage points of the 15% target. Bottleneck nodes (the top-10 betweenness nodes) were retained in every run. No community was reduced below its dynamic minimum of 1–3 nodes; the smallest retained community in the Australia run held 2 nodes (community 10, an L12-L24 cluster of 4 raw nodes).

6. Discussion

6.1 Why penalty-weighted multi-algorithm selection matters

Naive modularity maximisation is the standard practice in community-detection toolkits and is the implicit default in most circuit-analysis workflows. Our validation shows that on attribution graphs, naive modularity selects Greedy Modularity in two of three cases despite Greedy producing partitions that are clearly degenerate from an interpretability standpoint (3 communities, one or more spanning >30% of the graph). The penalty weights `−0.05 × n_oversized − 0.001 × n_singletons` add a small but reliable cost to such partitions while leaving competitive partitions in the lead. This is consistent with the broader observation in the multi-algorithm validation of our companion paper (Lawrence & Krampis, 2026, §5.19), in which mean cross-algorithm Jaccard agreement is only 0.363 — community boundaries are algorithm-dependent, but the *existence* of community structure is robust.

6.2 The semantic labelling gap

Across the three validation runs, only 4 of 39 communities (10.3%) received human-readable semantic labels; the remainder fell back to structural labels. This is a substantial limitation but is not a tool defect — it reflects the current sparsity of Neuronpedia's auto-generated SAE feature explanations, particularly for features that fire on technical vocabulary or non-English tokens. As Neuronpedia's annotation coverage improves (Bills et al., 2023; Templeton et al., 2024), the labelling success rate should improve mechanically without requiring tool changes. The structural fallback ensures the tool never fails completely; users always receive a layer-range and node-count description as a minimum.

6.3 Cantor pairing as a hidden API contract

The cantor-pairing transformation in §3.8 is, to our knowledge, undocumented in the Neuronpedia public API documentation. We discovered it through trial: graphs that validated successfully on `https://neuronpedia.org/graph/validator` and uploaded successfully to the viewer nonetheless rendered with broken feature dashboards (clicking a node produced a 404 from the feature lookup endpoint). Inspection of working graphs hosted by other users revealed that their `feature` fields were Cantor-paired integers rather than raw circuit-tracer feature IDs. This is a clear case of a tool that codifies undocumented platform behaviour to spare future users the same debugging cycle.

6.4 Comparison to existing tools

Feature	Supernode Detector	Manual Louvain	Neuronpedia web UI	ACDC (Conmy et al.)
Multi-algorithm selection with penalty	Yes	No	No	N/A (different goal)
Recursive splitting of oversized communities	Yes	No	No	N/A
Influence-based trimming	Yes	No	No	N/A
Automatic semantic labelling	Yes	No	No	No
Bottleneck pinning	Yes	No	Manual	N/A
Cantor-pairing transformation	Yes	No	N/A	N/A
Programmatic upload	Yes	N/A	Manual web form	N/A
Batch processing	Yes	No	No	Limited
Local model required	No	No	No	Yes (GPU)
Output format	Annotated JSON for viewer	Edge list / partition dict	N/A	Edge subset

The tool's value is integration. Each individual capability above exists in some form: NetworkX provides Louvain and centrality, Neuronpedia provides graph generation and feature explanations, ACDC and attribution patching provide alternative circuit-discovery methods. Supernode Detector composes these into a single command optimised for the workflow of "I want to share an interpretable interactive view of this prompt's circuit."

6.5 When to use which tool

For circuit-discovery research that requires interventional rigour (e.g., proving a circuit is necessary for a behaviour), ACDC, attribution patching, or causal scrubbing remain the appropriate methods. They produce edge-subset circuits with statistical guarantees but do not produce shareable interactive visualisations.

For exploratory analysis or for sharing results with non-specialist audiences (reviewers, collaborators, students), Supernode Detector is more directly suited. It consumes the same Neuronpedia graphs that researchers were going to inspect anyway, and produces a publishable artefact rather than an internal data structure.

The tool is also useful as part of a longer pipeline: in our companion paper, supernode detection serves as an annotation layer on top of traceback graphing, allowing per-circuit results to be communicated as labeled communities rather than as 1,200-node feature lists.

7. Limitations

1. **Single-model invocations.** Each pipeline run processes one model; cross-model comparison requires running the tool twice and comparing outputs manually. The companion paper performs such comparison externally.

2. **Semantic-labelling coverage.** As discussed in §6.2, only 10–15% of communities currently receive human-readable labels in the validation set. The tool exposes a `--rate-delay` flag for users who want to throttle API calls; otherwise rate limiting is the limiting factor on label-call throughput.

3. **Algorithm-dependent partitions.** Multi-algorithm validation in the companion paper shows mean cross-algorithm Jaccard of 0.363. Penalty weighting addresses the worst pathologies (oversized and fragmented partitions) but does not eliminate the underlying degeneracy; users should treat individual community boundaries as one valid partition among many.

4. **No interactive preview.** The current CLI emits the annotated JSON without an in-tool preview step; users who want to rename communities before upload must edit the JSON manually. A web UI for preview-and-rename is a planned future enhancement.

5. **Qwen support is partial.** Qwen3-4B graphs are generated and clustered correctly, but Neuronpedia's feature explanation API does not yet support Qwen SAEs, so semantic labels fall back to structural descriptions for all Qwen communities.

6. **Cantor-pairing assumes a 16,384-feature SAE.** The transformation hard-codes the modulus 16,384 to match `gemmascope-transcoder-16k` and the equivalent Qwen SAE. Users targeting SAEs with different dictionary sizes will need to adjust the constant in `MODEL_CONFIGS`.

8. Conclusion

We have presented Supernode Detector, an open-source command-line tool that takes a text prompt and produces an annotated, publishable Neuronpedia graph in a single command. The tool composes Neuronpedia's existing Circuit Tracer and feature-explanation APIs with three contributions of its own: a penalty-weighted multi-algorithm community selector that prefers interpretable partitions over nominally higher-modularity but degenerate ones; an influence-based trimming step that reduces a 1,200-node graph to ~170 retained nodes while preserving critical routing; and a cantor-paired export with three-step programmatic upload that codifies an otherwise-undocumented Neuronpedia API contract. Validation on three factual-recall prompts spanning chemistry, geography, and history demonstrates consistent algorithm selection, target trimming ratios within 2 percentage points of the 15% target, and successful end-to-end upload in under two minutes per prompt. The tool is positioned as a bridge between low-level circuit-discovery methods and the human-readable interactive visualisations that researchers and reviewers actually consume. Source, documentation, and validation outputs are available at `github.com/J-Lawrence10/autocircuit` under the MIT license.

Acknowledgments

We thank the Neuronpedia team (Decode Research) for hosting the Circuit Tracer API and SAE feature dashboards on which this tool depends, and the Gemma Scope (Google DeepMind) and Qwen SAE teams for releasing the underlying sparse autoencoders.

References

Bills, S., Cammarata, N., Mossing, D., Tillman, H., Gao, L., Goh, G., Sutskever, I., Leike, J., Wu, J., & Saunders, W. (2023). Language models can explain neurons in language models. OpenAI.

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, P10008.

Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., McLean, B., Burke, J. E., Hume, T., Carter, S., Henighan, T., & Olah, C. (2023). Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. *Transformer Circuits Thread*.

Conmy, A., Mavor-Parker, A. N., Lynch, A., Heimersheim, S., & Garriga-Alonso, A. (2023). Towards Automated Circuit Discovery for Mechanistic Interpretability. *NeurIPS 2023*.

Cunningham, H., Ewart, A., Riggs, L., Huben, R., & Sharkey, L. (2024). Sparse Autoencoders Find Highly Interpretable Features in Language Models. *ICLR 2024*.

Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., & Olah, C. (2021). A Mathematical Framework for Transformer Circuits. *Transformer Circuits Thread*.

Heimersheim, S., & Janiak, J. (2023). A circuit for Python docstrings in a 4-layer attention-only transformer. LessWrong.

Kramár, J., Lieberum, T., Shah, R., & Nanda, N. (2024). AtP*: An efficient and scalable method for localizing LLM behaviour to components. arXiv:2403.00745.

Lawrence, J., & Krampis, K. (2026). Cross-Domain Circuit Analysis of Factual Knowledge Retrieval in Large Language Models. Companion paper, this repository.

Lin, J. (2023). Neuronpedia: Interactive Reference and Tooling for Analyzing Neural Networks. https://neuronpedia.org

Lindsey, J., Gurnee, W., Ameisen, E., Chen, B., Pearce, A., Turner, N. L., Citro, C., Abrahams, D., Carter, S., Hosmer, B., Marcus, J., Sklar, M., Templeton, A., Bricken, T., McDougall, C., Cunningham, H., Henighan, T., Jermyn, A., Jones, A., Persic, A., Qi, Z., Thompson, T. B., Zimmerman, S., Rivoire, K., Conerly, T., Olah, C., & Batson, J. (2025). On the Biology of a Large Language Model. *Transformer Circuits Thread*. https://transformer-circuits.pub/2025/attribution-graphs/biology.html

Meng, K., Bau, D., Andonian, A., & Belinkov, Y. (2022). Locating and Editing Factual Associations in GPT. *NeurIPS 2022*.

Nanda, N. (2023). TransformerLens. https://github.com/TransformerLensOrg/TransformerLens

Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., & Carter, S. (2020). Zoom In: An Introduction to Circuits. *Distill*.

Syed, A., Rager, C., & Conmy, A. (2023). Attribution Patching Outperforms Automated Circuit Discovery. arXiv:2310.10348.

Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., Pearce, A., Citro, C., Ameisen, E., Jones, A., Cunningham, H., Turner, N. L., McDougall, C., MacDiarmid, M., Freeman, C. D., Sumers, T. R., Rees, E., Batson, J., Jermyn, A., Carter, S., Olah, C., & Henighan, T. (2024). Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. *Transformer Circuits Thread*.

Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9, 5233.

Wang, K., Variengien, A., Conmy, A., Shlegeris, B., & Steinhardt, J. (2023). Interpretability in the Wild: A Circuit for Indirect Object Identification in GPT-2 Small. *ICLR 2023*.